

CSS, Bootstrap & JavaScript — Answer Key

Answers to 10 questions on web development fundamentals

1. What is the CSS Box Model?

The CSS Box Model describes how every HTML element is represented as a rectangular box made up of four layers, counted from the inside out:

- **Content** — the actual text, image, or other content inside the element.
- **Padding** — transparent space between the content and the border; it pushes the border outward from the content.
- **Border** — a line that wraps around the padding and content.
- **Margin** — transparent space outside the border that separates the element from neighbouring elements.

By default, an element's total width is: content width + left/right padding + left/right border (this is the **content-box** model). Setting `box-sizing: border-box;` makes the specified width include padding and border, which is why it is the most commonly used setting in real projects.

```
div {
  width: 200px;
  padding: 20px;
  border: 5px solid #333;
  margin: 10px;
  box-sizing: border-box; /* width stays 200px total */
}
```

2. What is a Media Query? Explain with a sample code.

A media query is a CSS3 feature that applies a block of styles only when certain conditions about the viewport or device are true — most commonly the screen width. It is the core technique behind **responsive web design**, allowing a single stylesheet to adapt the layout for phones, tablets, and desktops.

```
/* Default styles for desktop */
.container {
  display: flex;
  width: 100%;
}

/* Applies only when the viewport is 768px wide or less */
@media (max-width: 768px) {
  .container {
    flex-direction: column;
  }
  body {
    font-size: 14px;
  }
}
```

Here, the layout switches from a row-based flex container to a stacked column layout automatically once the screen shrinks to tablet size or smaller.

3. What are the features of Bootstrap?

- **Responsive Grid System** — a flexible 12-column grid that adapts layouts across screen sizes.
- **Pre-styled Components** — ready-made navbars, cards, modals, carousels, buttons, forms, alerts, etc.
- **Responsive Utilities** — helper classes for spacing, alignment, display, and visibility at different breakpoints.
- **Cross-browser Compatibility** — consistent rendering across modern browsers.
- **Mobile-first Design** — styles are written for small screens first and scaled up with media queries.
- **Customizable Themes** — Sass variables and build tools allow colors, fonts, and spacing to be customized.
- **JavaScript Plugins** — built-in interactive components (dropdowns, tooltips, modals) that don't need extra libraries.
- **Extensive Documentation** and a large community/ecosystem of templates and plugins.

4. Write an example of a basic grid structure in Bootstrap.

Bootstrap's grid is based on containers, rows, and columns. A row is divided into 12 units, and columns specify how many of those units they occupy at a given breakpoint.

```
<div class="container">
  <div class="row">
    <div class="col-md-4">Column 1</div>
    <div class="col-md-4">Column 2</div>
    <div class="col-md-4">Column 3</div>
  </div>
</div>
```

This creates three equal-width columns (each taking 4 of the 12 grid units) on medium and larger screens. On smaller screens, they automatically stack vertically since no smaller-breakpoint class is specified.

5. What are CSS Selectors?

CSS selectors are patterns used to select and target the HTML elements you want to style. They are the part of a CSS rule that comes before the curly braces. Common types include:

- **Universal selector** (*) — selects all elements.
- **Element/type selector** (p, div) — selects all elements of that tag.
- **Class selector** (.classname) — selects elements with a given class.
- **ID selector** (#idname) — selects the single element with that ID.
- **Attribute selector** ([type="text"]) — selects elements with a given attribute/value.
- **Descendant selector** (div p) — selects elements nested inside another.
- **Child selector** (div > p) — selects direct children only.

- **Pseudo-class selectors** (:hover, :first-child) — select elements in a particular state or position.
- **Pseudo-element selectors** (::before, ::after) — style a specific part of an element.

6. What are CSS Backgrounds? List the properties.

CSS background properties control the background area of an element — its color, image, positioning, and repetition behaviour. Key properties include:

- `background-color` — sets a solid background color.
- `background-image` — sets an image as the background.
- `background-repeat` — controls whether/how the image repeats (repeat, no-repeat, repeat-x, repeat-y).
- `background-position` — sets the starting position of the background image.
- `background-size` — sets the size of the background image (e.g. cover, contain, or explicit dimensions).
- `background-attachment` — determines if the background scrolls with the page or stays fixed.
- `background-origin` — defines the positioning area of the background image.
- `background-clip` — defines how far the background extends (into padding, border, etc.).
- `background` — a shorthand property to set all of the above in one line.

```
body {
  background: url("bg.jpg") no-repeat center center / cover fixed;
}
```

7. How can we overlap elements in CSS?

Elements are overlapped mainly using **positioning** together with the **z-index** property:

- Set `position`: `relative`, `absolute`, `fixed`, or `sticky` on the elements so they can be moved out of normal document flow.
- Use `top`, `left`, `right`, `bottom` to place one element on top of another.
- Use `z-index` (an integer) to control the stacking order — elements with a higher `z-index` appear above elements with a lower one. `z-index` only works on positioned elements.
- Negative margins or CSS Grid/Flexbox with overlapping grid areas can also be used to make elements overlap.

```
.container { position: relative; }

.back {
  position: absolute;
  top: 0; left: 0;
  z-index: 1;
}

.front {
  position: absolute;
  top: 20px; left: 20px;
  z-index: 2; /* sits above .back */
}
```

}

8. What are the different positioning properties in CSS?

The `position` property defines how an element is placed in the document. Its values are:

- **static** — the default; the element follows normal document flow and `top/left/right/bottom` have no effect.
- **relative** — positioned relative to its own normal position; offsetting it does not affect surrounding elements.
- **absolute** — removed from normal flow and positioned relative to the nearest positioned ancestor (or the document if none exists).
- **fixed** — removed from normal flow and positioned relative to the browser viewport; stays in place while scrolling.
- **sticky** — behaves like `relative` until the viewport scrolls past a threshold, then behaves like `fixed` within its containing block.

9. What is ECMAScript?

ECMAScript (ES) is the standardized scripting language specification on which JavaScript is based. It is maintained by ECMA International (Technical Committee 39 / TC39) and defines the core syntax, types, statements, operators, and built-in objects that a compliant scripting language must implement. JavaScript is the most widely known implementation of ECMAScript, but other languages such as JScript and ActionScript are also based on it.

ECMAScript has evolved through numbered/yearly editions — for example ES5 (2009), and ES6 / ES2015, which introduced major features like `let/const`, arrow functions, classes, template literals, and promises. Since ES2015, new editions are released annually (ES2016, ES2017, and so on).

10. Difference between `var`, `let` and `const`; and the Temporal Dead Zone

Feature	<code>var</code>	<code>let</code>	<code>const</code>
Scope	Function-scoped	Block-scoped	Block-scoped
Re-declaration	Allowed	Not allowed	Not allowed
Re-assignment	Allowed	Allowed	Not allowed
Hoisting	Hoisted & initialized as undefined	Hoisted but not initialized	Hoisted but not initialized
Temporal Dead Zone	No	Yes	Yes

var is function-scoped (or globally scoped if declared outside a function), can be re-declared and re-assigned freely, and is hoisted to the top of its scope with its value initialized to `undefined` — so it can be accessed (as `undefined`) before the line it's declared on.

let is block-scoped (confined to the nearest { }), can be re-assigned but not re-declared in the same scope, and is used for variables whose value will change.

const is also block-scoped, but it cannot be re-assigned after its initial value is set (for objects and arrays, the reference is constant, though their contents can still be mutated).

Temporal Dead Zone (TDZ): This is the period between the start of a block and the line where a `let` or `const` variable is actually declared. Although these variables are technically hoisted to the top of their block, they are not initialized during this period — accessing them before their declaration throws a `ReferenceError` instead of returning `undefined`. This behaviour helps catch bugs caused by using a variable before it is meaningfully assigned.

```
console.log(a); // undefined (var is hoisted & initialized)
var a = 10;
```

```
console.log(b); // ReferenceError: Cannot access 'b' before initialization
let b = 20;      // b is in the Temporal Dead Zone until this line runs
```